Öğr.Gör. Mustafa GÖKMEN
gokmen@selcuk.edu.tr

Nesneye Yönelik Programlama

Sınıf & Nesne

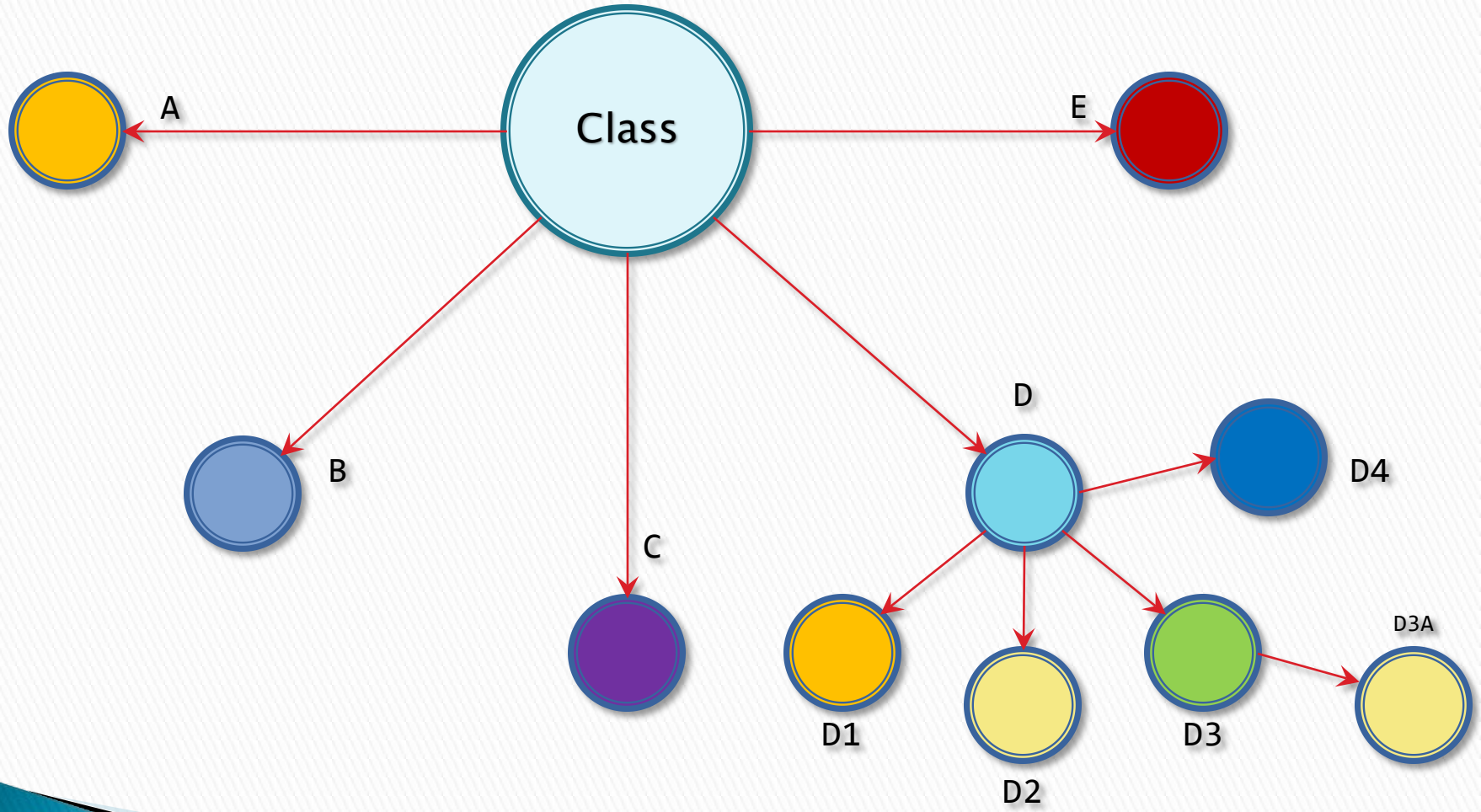
Sınıf ve Nesne

- Nesne, insan zihninin algılayabildiği özellik ve davranışlara sahip, fiziksel olan ya da olmayan herhangi bir olgudur.
- Yazılımda her tip nesne için, verisi ile davranışını bir paket haline getirip sarmalayan (encapsulation) ve adına genelde sınıf (class) denen kalıplar (şablonlar) oluşturulur.

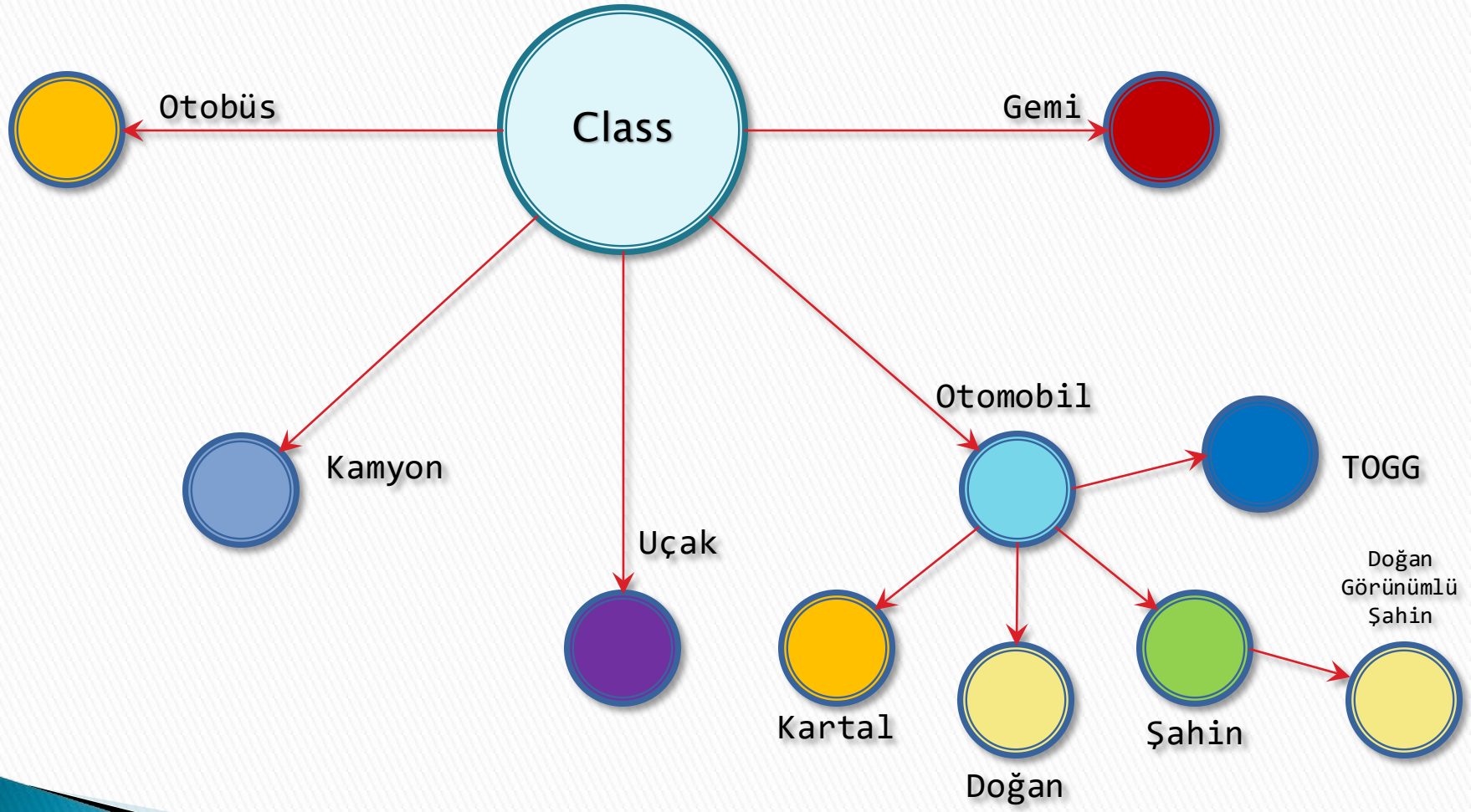
Sınıf = Veri + Davranış

- Nesneler, şablonlardan yani sınıflardan üretilmiş çalışma zamanı yapılarıdır.
- Nesneler gerçek dünyadaki varlıkları ya da olguları ifade ederler.
  - Nesnelerin özellikleri gerçek varlıkların özelliklerine, davranışları ise gerçek varlıkların birbirleriyle olan iletişimine karşılık gelir.
- Java'da basit veri tipleri hariç her şey, bütün tipler, bir nesnedir.

Sınıf ve Nesne



Sınıf ve Nesne



Sınıf ve Nesne

- *Sınıf*, o sınıfa ait olan bütün varlıkların ortak özellik ve davranışlarını ifade eden *bir tanımdır*.

- *Nesne;* ait olduğu sınıftan gelen özelliklerin değerlerinin belli olduğu, sınıfınca tanımlanmış olan davranışları nasıl sergilediğinin bilindiği, *somut olarak var ve biricik kimliği olan bir varlıktır*.
  - Aynı sınıfa ait bir çok farklı nesne söz konusu olabilir ve aynı sınıfa ait olan nesnelerin hepsinde, nesnenin ait olduğu sınıftan gelen (inherited) özellik ve davranışlar bulunmaktadır.
  - Her nesne; sınıfına ait özelliklerinin değerlerini ve davranışlarını farklı olarak kullanıyor olabilir.
  - Her nesnenin, kendisini işaret eden benzersiz bir kimliği vardır;
    - ✓ Öğrenci Numarası, Sicil Numarası, Araç Plakası vb. gibi.

Sınıf ve Nesne

Sınıf ve Nesne Örnekleri:

- Kalem Sınıfı:
  - Bütün kalemlerin yazı yazma davranışı, renk ve uç kalınlığı gibi özellikleri vardır ve bunlar "*kalem*" sınıfını ifade eden tanımlardır.
  - Dolma kalem, Kurşun kalem, Asetat kalemi vb. ise bu sınıfa ait ortak özelliklere sahip olmakla birlikte kendisine ait özellikleri de olan farklı nesnelerdir.
- Otomobil Sınıfı:
  - Bütün otomobillerin hızlanma, yavaşlama, dönüş vb. gibi davranışları, renk, motor, lastikler, kapılar, koltuklar vb. gibi özellikleri vardır.
  - Ancak her modelde, bu eylem ve özellikler o modele özgü olarak değişiklik gösterir.
    - ✓ Mesela her otomobil, 0'dan 100 km. hıza farklı zamanlarda ulaşacaktır ve her otomobil dört kapılı değildir.

Örnek: Nesne Oluşturma

- Sınıfın bir tanım, nesnenin ise o tanıma göre oluşturulmuş fiziksel bir varlık olduğunu tekrar hatırlayalım.
- Nesne oluşturulmadığı sürece bellekte fiziksel olarak oluşturulmuş bir varlık yoktur. Sınıfın nitelikleri için bellekten yer ayrılmamıştır. Örneğin, Vakit sınıfına ait nesne oluşturulmadığı sürece saat, dakika ve saniye nitelikleri yalnızca birer tanımdır. Nesne fiziksel olarak oluşturulduğu andan itibaren ise bu niteliklerin her birisi için bellekten yer ayrılacaktır.
- Herhangi bir sınıfa ait bir nesne oluşturulabilmesi için new opratörü kullanılır. new, belirtilen sınıfa ait bir nesne oluşturup bu nesnenin bellekteki adresini (referansını) döndürür.
- Aynı satırda hem sınıfa ait nesnenin adresini tutabilecek bir değişken tanımı yapılmış, hem de sınıftan bir nesne oluşturulup, nesnenin adresi tanımlanan bu değişkenin içine atanmıştır. `Vakit` tipinde `yeni Vakit` nesnesi oluşturup adresini vkt adlı referans değişkene atamak için gereken ifade şudur:

```
Vakit vkt = new Vakit();
```

Örnek: Vakit Sınıfı

- Örnek olarak *Vakit* adlı bir *sınıf* (class) tanımı yapalım. Sınıfımızın *saat*, *dakika* ve *saniye* adlı *özellikleri* ile *ilerlet()* ve *vakitYaz()* adlı *davranışları* olsun. *saat*, *dakika* ve *saniye* özelliklerini, sınıfa ait bir nesnenin durumunu oluşturacak tamsayı değerler, *ilerlet()* o nesneye air değerleri bir saniye sonrasına ilerleten bir davranış ve *vakitYaz()* da çağrıldığı andaki özellikleri ekrana yazdıran bir davranış olsun.

- Sınıf tanımı yapılırken, *özellikler* sınıfın içine *nitelik* olarak, *davranışlar* ise *metod* olarak kodlanır.

- *Özellik (property)* NYP açısından, nesnenin durumunu oluşturan, nesnenin ait olduğu sınıfın içine kodlanmış her bir değişkene verilen isimdir. Özellik, Java açısından *nitelik (attribute)* olarak adlandırılır.

- *Davranış (behaviour)* NYP açısından, nesnenin durumunda değişiklik yapabilen, nesnenin ait olduğu sınıfın içine kodlanmış her bir işleve verilen isimdir. Davranış, Java açısından *metod (method)* olarak adlandırılır.

Örnek: Vakit Sınıfı

```
public class Vakit {
  public int saat;
  public int dakika;
  public int saniye;

  public void ilerlet() {
    saniye++;

    if (saniye == 60) {
      saniye = 0;
      dakika++;
      if (dakika == 60) {
        dakika = 0;
        saat++;
        if (saat == 24) {
          saat = 0;
        }
      }
    }
  }

  public void vakitYaz() {
    System.out.println("Zaman : %0,2d:%0,2d:%0,2d\n", saat, dakika, saniye);
  }
}
```

Örnek: VakitApp Sınıfı

`main()` metodunu *Vakit* sınıfının içine yazabiliriz. Ancak bu sınıf kendi başına düşünüldüğünde, çalıştırılabilir bir sınıf olmak zorunda değildir. Bu sınıf, yalnızca zaman bilgisini ifade edebilen nesneler oluşturulabilmesi için yazılmış bir tanımdır. Başka bir deyişle, sınıfa ait kodların içinde bulmayı beklediğimiz yöntemler yalnızca bu sınıfa ait davranışlar olmalıdır. Öyleyse sırf örneğimizi çalıştırabilmek için sınıfın içine, aslında kendi davranışları ile ilgisi olmayan `main()` metodunu yazmak güzel bir yaklaşım olmayacaktır.

`main()` metodu çalıştırılabilir bir program elde etmek üzere yazılan bir yöntem olduğundan ve Java'da bütün kodların mutlaka bir sınıfın içine yazılması zorunlu olduğundan, bu yöntemi VakitApp adlı bir sınıfın içine yazmak daha mantıklı olacaktır.

```
public class VakitApp {
  public static void main(String[] args) {
    Vakit vkt = new Vakit();

    vkt.saat = 23;
    vkt.dakika = 59;
    vkt.saniye = 59;

    vkt.vakitYaz();
    vkt.ilerlet();
    vkt.vakitYaz();
  }
}
```

```
Zaman: 23:59:59
Zaman: 00:00:00
```

Öğr.Gör. Mustafa GÖKMEN
gokmen@selcuk.edu.tr

Nesneye Yönelik Programlama

Input & Output

Input / Output

- ✓ `System` sınıfı varsayılan paket olan *java.lang* paketi içindedir ve üç tane sınıf değişkeni (`err`, `in`, `out`) vardır.

- ✓ `out` değişkeni standart çıkış (ekran) akımıdır. `out` değişkeni `PrintStream` tipindedir ve *java.io* paketindeki `PrintStream` sınıfının metotlarını (`print()`, `println()`, `printf()`) kullanabilir.

```
java.io
Class PrintStream

java.lang.Object
    java.io.OutputStream
        java.io.FilterOutputStream
            java.io.PrintStream
```

- ✓ `in` değişkeni standart giriş (genellikle klavye) akımıdır.

- ✓ `err` değişkeni standart hata mesajları için kullanılan akımdır.

Scanner Sınıfı ve Veri Okuma

- ✓ Java konsol çıktıları için `System.out` ve girdileri için `System.in` kullanır.
- ✓ Console girdisi direk desteklenmediği için Scanner nesnesi `System.in` girdisini okumak için kullanılır (*java.util.Scanner* import edilmelidir).
- ✓ `Scanner` sınıfının yapıcısına `System.in` *adlı* `InputStream` kaynağı verilir (Standart Input Stream - Klavyeden Veri Girişi). Daha sonra oluşturduğumuz *scanner* nesnesi üzerinden `Scanner` sınıfının metotlarına erişilir.
- ✓ Bir scanner nesnesi girilen veriyi parçalarına (token) ayırır. Bu ayırma işleminde parça ayracı varsayılan olarak beyaz alanlardır (boşluk, tab, yeni satır vb). Ancak, istenirse parça ayracı farklı tanımlanabilir. Scanner nesnesi ayırdığı parçaları ait olduğu veri tipine dönüştürür. Bunu yapan farklı `next()` metodları vardır.

Aşağıdaki kodlar, klavyeden (`System.in`) girilen tamsayıları okur.

```
Scanner kb = new Scanner(System.in);
int var = kb.nextInt();
```

Aşağıdaki kodlar, String ifadeyi okuyup sonraki satırı ekrana yazdırır.

```
String userName = "Dr. Ruhi BUNALIM";
Scanner kb = new Scanner(userName);
System.out.println("Merhaba, " + kb.nextLine());
```

Scanner Sınıfı ve Veri Okuma

`hasNext()`, `hasNextInt()`, `hasNextByte()` vb. gibi çeşitleri bulunur. Bulunduğu satırdan sonra okunacak girdi içerisinde, hâlâ okunabilecek string, int, byte vs. verilerinin bulunup bulunmadığını kontrol eder. Bulursa `true`, bulamazsa `false` değerini döndürür.

```
Scanner sc = new Scanner("Merhaba Dünya");
System.out.println("" + sc.hasNext());
System.out.println("" + sc.nextLine());
System.out.println("" + sc.hasNext());
sc.close();
```

```
true
Merhaba Dünya
false
```

```
Scanner sc = new Scanner("Merhaba Dünya");
System.out.println("" + sc.hasNext());
System.out.println("" + sc.next());
System.out.println("" + sc.hasNext());
sc.close();
```

```
true
Merhaba
true
```

Scanner Sınıfı ve Veri Okuma

```java
import java.util.Scanner;

public class ScanDemo1 {
  public static void main(String[] args) {
    String userName = "Mustafa Kamil Zorti";
    Scanner kb = new Scanner(userName);
    System.out.printf("Merhaba, " + kb.next());

    while (kb.hasNext()) {
      System.out.print(" " + kb.next());
    }
    System.out.println("!.. Nasılsınız?");
    kb.close();
  }
}
```

```
Merhaba, Mustafa Kamil Zorti... Nasılsınız?
```

```java
import java.util.Scanner;

public class ScanDemo2 {
  public static void main(String[] args) {
    Scanner kb = new Scanner(System.in);
    System.out.print("Adınız? : ");
    String uName = kb.nextLine();
    kb.close();

    Scanner str = new Scanner(uName);
    System.out.print("Merhaba, ");
    do {
      System.out.print(str.next());
      if (str.hasNext()) {
        System.out.print(" ");
      } else {
        break;
      }
    } while (str.hasNext());
    System.out.print("!.. Nasılsınız?");
    str.close();
  }
}
```

```
Adınız? : Mustafa Kamil Zorti
Merhaba, Mustafa Kamil Zorti!.. Nasılsınız?
```

Scanner Sınıfı ve Veri Okuma

```java
import java.util.Scanner;

public class ScanDemo3 {
  static Scanner kb = new Scanner(System.in);
  String         name;

  public static void main(String[] args) {
    System.out.printf("Yeni bir metin giriniz : ");
    String str1 = kb.nextLine();

    System.out.printf("Bir metin daha giriniz : ");
    String str2 = " " + kb.nextLine();

    System.out.println(str1.concat(str2).toUpperCase());
  }
}
```

```
Yeni bir metin giriniz : dal sarkar kartal kalkar
Bir metin daha giriniz : kartal kalkar dal sarkar
DAL SARKAR KARTAL KALKAR KARTAL KALKAR DAL SARKAR
```

Scanner Uygulaması (v1.0)

```java
import java.text.SimpleDateFormat;
import java.util.*;

public class DogumYili1 {
  static Scanner kb = new Scanner(System.in);
  public static void main(String[] args) {
    Date             date     = new Date();
    SimpleDateFormat sdf      = new SimpleDateFormat("YYYY");
    String           userName = "";
    System.out.printf("Ad-Soyad ve Yaşınız?: ");

    do {
      userName += kb.next();
      if (!kb.hasNextInt()) {
        userName += " ";
      }
    } while (!kb.hasNextInt());

    Integer          birth = Integer.parseInt(sdf.format(date)) - kb.nextInt();
    System.out.printf("%s, %d doğumludur.", userName.toUpperCase(), birth);
  }
}
```

```
Ad-Soyad ve Yaşınız?: mustafa kamil zorti 36
MUSTAFA KAMİL ZORTİ, 1987 doğumludur.
```

Scanner Uygulaması (v2.0)

```java
import java.util.Calendar;
import java.util.Scanner;

public class DogumYili2 {
  static Scanner kb = new Scanner(System.in);
  public static void main(String[] args) {
    String userName = "";
    System.out.printf("Ad-Soyad ve Yaşınız?: ");

    do {
      userName += kb.next();
      if (!kb.hasNextInt()) {
        userName += " ";
      }
    } while (!kb.hasNextInt());

    Calendar cal   = Calendar.getInstance();
    Integer  birth = cal.get(Calendar.YEAR) - kb.nextInt();
    System.out.printf("%s, %d doğumludur.", userName.toUpperCase(), birth);
  }
}
```

```
Ad-Soyad ve Yaşınız?: mustafa kamil zorti 36
MUSTAFA KAMİL ZORTİ, 1987 doğumludur.
```